

Assignment 1

Due: October 2, 2026. 2355 hrs.

General instructions:

Fill in the solutions in the file **Assignment1.hs** (provided on Moodle) and submit.

Do **not** change the names and types of the functions in **Assignment1.hs**.

Do **not** import any libraries in your file.

Solve the problems your own. Make sure that the program compiles.

Enter your solutions in a file named **Assignment1.hs** and submit on Moodle.

1. An element of a list is said to be a *middle element* if there are equal number of elements to its left and right. Write a function **getMiddle :: [a] → Maybe a** which takes an input list and returns **Just x** if **x** is a middle element, and returns **Nothing** if the list does not have a middle element.

```
getMiddle [1, 2, 3]           = Just 2
getMiddle [1, 2, 3, 4]        = Nothing
getMiddle ['a']               = Just 'a'
getMiddle []                  = Nothing
getMiddle [1.0, 2.0, 3.0, 4.0, 5.0] = Just 3.0
```

2. Write a function **isPal :: String → Bool** which checks whether an input string is a palindrome.

```
isPal "racecar" = True
isPal "hello"   = False
isPal ""        = True
isPal "a"       = True
isPal "abba"    = True
```

3. Define a function **runs :: (a → a → Bool) → [a] → [[a]]** where **runs p xs** the list of *maximal segments* $[xs !! i, xs !! i+1, \dots, xs !! j]$ s.t. **p xs !! k xs !! k+1** is **True** for all $k \in \{i, \dots, j-1\}$.

```
runs (<) [1, 2, 3, 1, 2] = [[1,2,3],[1,2]]
runs (==) [1, 1, 2, 2, 2, 3] = [[1,1],[2,2,2],[3]]
runs (>) [5, 4, 3, 6, 5] = [[5,4,3],[6,5]]
runs (\_ _ → False) [1, 2, 3] = [[1],[2],[3]]
runs (\_ _ → True) [1, 2, 3] = [[1,2,3]]
```

4. Define a function `sortWords :: [String] → [String]` which sorts a list of words in dictionary order. Do **not** use the built-in `sort` function. You can use `<` to compare two strings according to dictionary order.

```
sortWords ["banana", "apple", "cherry"] = ["apple", "banana", "cherry"]
sortWords ["Zebra", "apple"]           = ["Zebra", "apple"]
sortWords []                           = []
sortWords ["a"]                         = ["a"]
sortWords ["b", "a", "c"]               = ["a", "b", "c"]
```

5. A matrix can be represented as a list of lists (assume the elements are of type `Int`). Write a function `isMatrix` which checks whether a list of lists represents a matrix. Write another function `myTranspose` which computes the transpose of a matrix.

```
isMatrix :: [[Int]] → Bool
isMatrix [[1,2],[3,4]]      = True
isMatrix [[1,2],[3]]        = False
isMatrix [],[]              = True
isMatrix [[1]]               = True
isMatrix [[1,2,3],[4,5,6]]  = True

myTranspose :: [[Int]] → [[Int]]
myTranspose [[1,2],[3,4]]    = [[1,3],[2,4]]
myTranspose [[1,2,3]]        = [[1],[2],[3]]
myTranspose [[1],[2],[3]]    = [[1,2,3]]
myTranspose [[1,2],[3,4],[5,6]] = [[1,3,5],[2,4,6]]
myTranspose [[1]]             = [[1]]
```